Polymarket Data User Guide

Supabase

A read-only Postgres connection is available for querying market metadata. No VPN required.

Connection

```
None
postgresql://readonly.merbqatlxbrcvtbnuaqe:<PASSWORD>                @aws-0-us-west-2.pool
er.supabase.com:6543/postgres
```

Schema

`markets`

Column	Type	Notes
`slug`	text PK	Unique market identifier
`title`	text	Human-readable question
`condition_id`	text	On-chain condition id
`event_slug`	text FK	Parent event (`events.slug`)
`start_date`, `end_date`	timestamptz	Scheduled open/close
`active`, `closed`, `archived`	boolean	Market status flags
`last_trade_price`, `best_bid`, `best_ask`	numeric	Top-of-book snapshot at last sync
`volume_clob`, `volume_24hr`, `volume_1wk`, `volume_1mo`	numeric	CLOB volume
`token_ids`	jsonb	Array of CLOB asset ids (YES then NO)
`outcomes`	jsonb	Outcome labels matching `token_ids` order

Column	Type	Notes
neg_risk	boolean	Outcomes are mutually exclusive

events

Column	Type	Notes
slug	text PK	Unique event identifier
title	text	Event title
ticker	text	Short ticker symbol
start_date, end_date	timestamptz	Event date range
active, closed, archived	boolean	Event status flags
neg_risk	boolean	All child markets are mutually exclusive

resolutions

One row per resolved market — who won, when, and at what price. Joins to markets.slug.

Column	Type	Notes
market_slug	text PK, FK	Joins to markets.slug
resolved_yes	boolean	true = Yes won, false = No won
price_at_close	numeric	Final YES-token price (Gamma's outcomePrices[0]); ≥0.99 or ≤0.01 by construction
resolved_at	timestamptz	When the market closed (Gamma's closedTime, falling back to endDate if missing)
uma_resolution_status	text	Gamma's umaResolutionStatus at

Column	Type	Notes
		sync time, e.g. `'resolved'`; `NULL` for older pre-UMA markets
`price_24h_before`	numeric	Not yet populated (needs ticker history)

Coverage note: the full history was backfilled as of 2026-10-05, and new rows are added automatically within ~15 minutes of a market closing (checked against the Gamma API on a schedule, not a one-time bulk import). A market with a past `end_date` and no row in `resolutions` is still open past its scheduled end (e.g. awaiting UMA resolution), not a gap in coverage.

Connecting

psycopg2:

```python
import psycopg2

con = psycopg2.connect(
    "postgresql://readonly.merbqatlxbrcvtbnuaqe:<PASSWORD>"
    "@aws-0-us-west-2.pooler.supabase.com:6543/postgres?sslmode=require"
)
```

SQLAlchemy (for pandas or ORM use):

```python
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg2://readonly.merbqatlxbrcvtbnuaqe:<PASSWORD>"
    "@aws-0-us-west-2.pooler.supabase.com:6543/postgres",
    connect_args={"sslmode": "require"},
)
```

DuckDB (no Postgres driver needed):

```python
import duckdb

duckdb.sql("INSTALL postgres; LOAD postgres;")
duckdb.sql("""
    ATTACH 'host=aws-0-us-west-2.pooler.supabase.com port=6543 dbname=postgres
             user=readonly.merbqatlxbrcvtbnuaqe password=<PASSWORD>
sslmode=require'
    AS pm (TYPE postgres, READ_ONLY);
""")
```

Query examples

Active markets sorted by weekly volume:

```python
import pandas as pd

pd.read_sql("""
    SELECT slug, title, best_bid, best_ask, volume_1wk, end_date
    FROM markets
    WHERE active = true
    ORDER BY volume_1wk DESC NULLS LAST
    LIMIT 50
""", con)
```

Markets closing within the next 7 days:

```python
pd.read_sql("""
    SELECT slug, title, end_date, best_bid, best_ask, volume_1wk
    FROM markets
    WHERE active = true
      AND end_date BETWEEN now() AND now() + interval '7 days'
    ORDER BY end_date
""", con)
```

All markets under a given event:

```python
pd.read_sql("""
    SELECT m.slug, m.title, m.best_bid, m.best_ask, m.volume_1wk
```

```
    FROM markets m
    JOIN events e ON e.slug = m.event_slug
    WHERE e.title ILIKE '%2026 midterm%'
    ORDER BY m.volume_1wk DESC NULLS LAST
""", con)
```

Get the CLOB asset ids for a market (needed to query the order book API):

```python
import json

cur = con.cursor()
cur.execute("SELECT token_ids, outcomes FROM markets WHERE slug = %s",
("will-trump-win-2024",))
row = cur.fetchone()
token_ids = json.loads(row[0])   # ["0xabc...1", "0xabc...2"]
outcomes  = json.loads(row[1])   # ["Yes", "No"]
asset_map = dict(zip(outcomes, token_ids))
# {"Yes": "0xabc...1", "No": "0xabc...2"}
```

Browse active events and their market counts:

```python
pd.read_sql("""
    SELECT e.slug, e.title, COUNT(m.slug) AS market_count,
           MAX(m.volume_1wk) AS top_market_volume
    FROM events e
    JOIN markets m ON m.event_slug = e.slug
    WHERE e.active = true
    GROUP BY e.slug, e.title
    ORDER BY top_market_volume DESC NULLS LAST
""", con)
```

Who won a resolved market, and who to analyze as the winning side:

```python
pd.read_sql("""
    SELECT m.slug, m.title, r.resolved_yes, r.price_at_close, r.resolved_at
    FROM resolutions r
    JOIN markets m ON m.slug = r.market_slug
    WHERE r.resolved_at > now() - interval '7 days'
```

```
    ORDER BY r.resolved_at DESC
""", con)
```

Order book data (S3 / CloudFront)

Processed order book files are stored in S3 and served via CloudFront. No authentication or API key required.

Base URL:

```
None
https://d1yyqbluk0oz0t.cloudfront.net
```

Two file types are available per hour:

Filename pattern	Description
`events-<hour>.parquet`	Deltas — one row per level change
`snap-<hour>.parquet`	Snapshot — full book state at the start of that hour

Delta schema (`events-*.parquet`):

Column	Type	Notes
`ts_ms`	int64	Event timestamp, epoch ms UTC
`asset_id`	string	CLOB token id
`market`	string	Condition id
`side`	string	`BUY` or `SELL`
`price`	float64	Price level
`size`	float64	New size at this level (`0.0` = level removed)

Snapshot schema (`snap-*.parquet`):

Column	Type	Notes
`ts_ms`	int64	Start of hour, epoch ms UTC
`asset_id`	string	CLOB token id
`market`	string	Condition id
`bids`	list of `{price, size}`	All bid levels, best first
`asks`	list of `{price, size}`	All ask levels, best first

Reconstructing the book at any time T

1. Load the snapshot for the hour at or before T (`snap-<hour>.parquet`)
2. Replay delta rows from that snapshot forward up to T

Query examples

Query deltas for one hour directly — no download needed:

```python
import duckdb

duckdb.sql("INSTALL httpfs; LOAD httpfs;")

duckdb.sql("""
    SELECT *
    FROM 'https://d1yyqbluk0oz0t.cloudfront.net/events-2026-05-30T21.parquet'
    WHERE asset_id = '0xabc...1'
    ORDER BY ts_ms
""").df()
```

Load a snapshot and replay deltas forward to reconstruct the book at a point in time:

```python
import pandas as pd
import duckdb

BASE = "https://d1yyqbluk0oz0t.cloudfront.net"

# Full book state at the start of the hour
```

```
snap = duckdb.sql(f"SELECT * FROM '{BASE}/snap-2026-05-30T21.parquet'").df()

# All level changes in that hour for one asset
deltas = duckdb.sql(f"""
    SELECT * FROM '{BASE}/events-2026-05-30T21.parquet'
    WHERE asset_id = '0xabc...1'
    ORDER BY ts_ms
""").df()
```

Query multiple hours at once with a list:

Python

```
hours = ["2026-05-30T21", "2026-05-30T22", "2026-05-30T23"]
urls  = [f"'{BASE}/events-{h}.parquet'" for h in hours]

duckdb.sql(f"""
    SELECT * FROM read_parquet([{', '.join(urls)}])
    WHERE asset_id = '0xabc...1'
    ORDER BY ts_ms
""").df()
```

Wallet trade history (S3 / CloudFront)

Per-trade wallet history, one row per fill, pseudonymized. Same base URL and access as the order book data above.

Files: `wallet_trades/trades-<date>.parquet` (one per calendar date the trade occurred on, not per capture date). Sorted by `market` within each file, so a `WHERE market = '...'` filter is cheap even without a separate partition scheme.

Schema:

Column	Type	Notes
`wallet`	string	Pseudonymized — `0x` + 40 hex chars, same shape as a real address but not traceable to one

Column	Type	Notes
`market`	string	Market slug (joins to `markets.slug`)
`time`	timestamp (UTC)	When the trade occurred
`side`	string	`BUY` or `SELL`
`outcome`	string	e.g. `Yes` / `No`
`price`	float64	
`size`	float64	Shares traded
`dollars`	float64	USD value of the fill

Pseudonymization: `wallet` is never the real on-chain address. Each real wallet maps to one random, consistent pseudo address.

Query example:

```python
import duckdb
duckdb.sql("INSTALL httpfs; LOAD httpfs;")

duckdb.sql("""
    SELECT wallet, sum(dollars) AS volume, count(*) AS trades
    FROM
'https://d1yyqbluk0oz0t.cloudfront.net/wallet_trades/trades-2026-10-03.parquet'
    WHERE market = 'will-coreweave-announce-bankruptcy-by-december-31-2027'
    GROUP BY wallet
    ORDER BY volume DESC
""").df()
```

Glob multiple dates the same way as the order book examples above: `read_parquet('wallet_trades/trades-2026-10-*.parquet')`.

Coverage notes

Time coverage

- Pipeline started capturing on **2026-05-30**.
- Polymarket doesn't replay historical WS data, so any captured gap is permanent.

Known gaps (WebSocket recorder was down):

Gap	**Duration**
2026-06-16T11 → 2026-06-19T19	80 hours
2026-08-25T00 → 2026-08-31T14	158 hours
2026-09-14T01 → 2026-09-14T15	14 hours

After each gap the recorder reconnected and sent fresh full-book snapshots for all active assets, so book state self-corrects from that point forward. The snapshot file for the first hour after a gap may reflect stale pre-gap book state; use the delta rows in that file (which begin with `book` reset events) as the authoritative baseline instead.

Wallet trade history coverage

The wallet trade history covers through **2026-10-05**, including 1.1M+ markets.